

1. Introduction au calcul haute-performance : optimisation et parallélisation

(→ Norbert KERN)

- Quantifier la performance
- Compiler avantageusement (premiers éléments) + Optimiser son code source
- Jouer sur la hardware : le cache
- Paralléliser (OpenMP, MPI)
- Analyser (finement) les performance
- Combiner C et Python

2. Algorithmes : parallélisation d'un code de Dynamique Moléculaire classique

(→ Simona ISPAS)

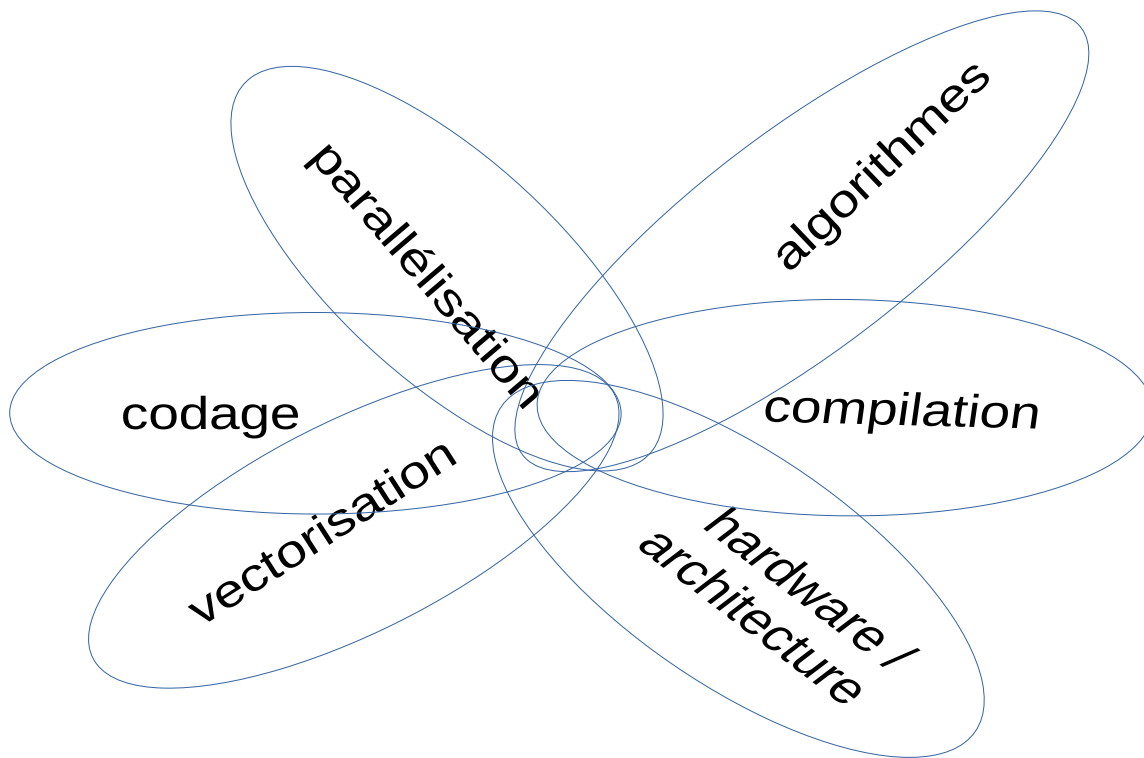
- Stratégies de parallélisation : décomposition atomique et spatiale
- Implémentation pratique et vérification

3. Mise en oeuvre : simulations ab initio

(→ Simona ISPAS)

- Théorie de la fonctionnelle de la densité (DFT)
- Dynamique moléculaire ab-initio
- Implémentation pratique (notions de base)

INTRODUCTION AU CALCUL HAUTE-PERFORMANCE : OPTIMISATION ET PARALLÉLISATION



Cependant :

**« We should forget about small efficiencies, say about 97% of the time:
premature optimization is the root of all evil »**

Donald Knuth

Rappel : étapes de résolution un problème par approche numérique

- i. Modélisation
- ii. Conception des algorithmes
- iii. Implémentation
- iv. Compilation
- v. Exécution
- vi. Exploitation/interprétation des données

(I) Modélisation :

c'est un métier ... pas ici ...

mais le choix peut impacter (si c'est trop coûteux c'est inutile; possibilité d'astuces; ...)

(II) Conception des algorithmes

Très important, surtout quand c'est mal fait ... penser à ces deux 'échelles' :

- algorithmes à l'**échelle globale** (choix d'un algorithme, agencement des étapes successives du code, etc)
→ *éliminer de faire des calculs coûteux évitables*
- implémentation à **échelle 'locale'** (programmer une opération, une boucle, une fonction)
→ *faire efficacement les calculs coûteux inévitables*

(III) Implémentation

Écrire le code source, éventuellement organisé en bibliothèques, sous-fichiers etc; documentation;
C (voir C++) ou Fortran

(IV) Compilation

Traduction du 'programme' dans un langage de programmation en 'programme' exécutable

Traduction du langage (plus ou moins) 'universel' au code binaire spécifique à l'architecture

Code (C, Fortran) → langage machine	→ exécutable binaire
printf("%ld",x) → movl \$8, %edx	→ 10111011010100001010
int \$0x80	01001000101110111010
...	10010100010101010101
	11101000101111010101
	...

Rem:

- les données, mais aussi le code exécutable, sont **binaires** !
- but d'un code : transformer des données (binaires);
ex: convertir cfg. initiale (00....110) en cfg. finale (01...101)

Ex : gcc truc.c && ./a.out

Astuce : le '&&' enchaîne la deuxième commande, comme si elle était séparée par ';', mais uniquement si la première commande réussit

En réalité, en plusieurs étapes :

- *pré-compilation* : transformation du code source, selon spécifications.

Ex (simple) :

```
g++ -DSILENT hello.C
```

→ le code est adapté en fonction de si le 'flag' **SILENT** est défini ou non

Rem: d'autres exemples à venir

- puis *compilation*, dont la première partie consiste à produire le *code assembleur*

Rem: on peut voir le code 'machine' (assembleur) : compiler avec l'option -S produit un fichier .s, qui contient ce code en clair

```
gcc -S hello.c && more hello.s
```

- fin de la compilation : fabrication d'un *binaire* exécutable (a.out par défaut)

```
gcc hello.c && ./a.out
```

Rem: plusieurs compilateurs (gnu, Intel, Portland, ...); suivent les normes du langage mais certaines interprétations divergentes peuvent exister sur des points précis

Ex: application des normes IEEE sur les nombres à virgule flottante), gestion du 'double zéro' des entiers, etc.

(V) Exécution

Chargement du binaire exécutable en mémoire, puis modification des données selon les algorithmes codés par application successive des instructions binaires.

Nécessite gestion des instructions (CPU) et des données (RAM)

(VI) Exploitation/interprétation des données

Peut être intégrée (logiciel '*monolithique*'), mais typiquement, en Sciences :

Agencement modulaire :

- production de configurations successives (executable)
- sauvegarde progressive sur disque
- analyse / interprétation '*post mortem*'

Avantages/inconvénients : lenteur de l'écriture; taille des données produites sur disque; facilité de scriptage; disponibilité des données pour analyse ultérieure modifiée/raffinée; impossible si besoin du résultat de l'analyse pour la suite de la simulation (ex: simulations à biais thermodynamique)

QUANTIFIER LA PERFORMANCE

Premier pas : quantifier, puis mesurer la ‘performance’

Qqs indicateurs pertinents :

- temps d’exécution → utilisateur impatient
- flops (floating point operations per second) → cpu
- iops (input/output operations per second) → i/o
- paquets/octetets par sec → réseau
- flops/Watt → consommation électrique
- flops/euro → budget
- scalabilité → calculs parallèles

Lequel utiliser ? Cela dépend ...

Quel indicateur pour les simulations atomistiques ? Cela dépend aussi ...

La base, en pratique : mesurer le temps de calcul pour l’exécution d’un certain code est toujours instructif !

Mesurer le temps d’exécution (linux)

Rem: cela est possible aussi depuis un code même (bibliothèques en C, même Python)

Ici: commande **time** :

Ex:

```
$ time sleep 5
real 0m5,002s
user 0m0,002s
sys 0m0,000s
```

‘wall time’ vs. ‘user time’ vs. ‘cpu time’

- real = ‘wall time’ = horloge : chronomètre
- user = ‘cpu time’ : temps utilisé pour calculer
(rem: cumulé si processeurs multiples)
- sys = ‘system time’ : appels de système
(rem: cumulé sur sous-processus comme ‘forks’ etc.)

Rem: cpu < wall si processeur simple, mais pas nécessairement sur multi-processeur

À noter aussi une subtilité : **time** est une commande de terminal, mais il existe aussi un exécutable **/usr/bin/time** (attention à la confusion involontaire !). Chacun a des avantages, mais le dernier peut être pratique ici car il permet de reconfigurer le format dans lequel le résultat est rendu.

Complexité algorithmique

L'**idée** est d'analyser comment le besoin de ressources dépend de la taille du 'système' (taille d'un tableau, chiffres retenus pour représenter un entier, longueur d'une chaîne de caractères, nombre de particules, etc).

- on stipule une fonction $f(N)$ décrivant comment les ressources nécessaires dépendent de la taille du système N
- ici, on considère le temps de calcul en fonction de la taille du système (mais des variantes caractérisent la mémoire nécessaire, etc.)
- on peut se baser soit sur le scénario le plus défavorable, soit sur une moyenne

ici donc :

taille du système $N \rightarrow$ temps de calcul $f(N)$

Pour une **opération simple** (comme par ex. l'addition, la multiplication ...,)

- nécessite un certain temps de calcul
- ce temps varie d'un simple facteur d'une machine à l'autre
- la répétition de telles opérations multiplie simplement le temps total de calcul

La complexité caractérise donc directement l'algorithme utilisé

Exemples :

- l'addition est une opération de complexité $O(N)$ par rapport à au nombre N de chiffres retenus : on somme sur N chiffres, de manière identique (somme de nombres entre 0 et 9, la retenue, et il faut gérer la nouvelle retenue)
- la multiplication est un algorithme de complexité $O(N^2)$ si on l'effectue comme c'est enseigné à l'école.

Rem: il existe des algorithmes qui la ramènent, dans le meilleur des cas, à $O(N \cdot \log(N))$

Question/Exercice : quelle est la complexité algorithmique de la multiplication de deux matrices de dimension $N \times N$?

Pour des **opérations plus complexes**, malgré le fait que le temps d'exécution peut varier en fonction des circonstances (charge de calcul instantanée du processeur, taux d'occupation de la mémoire vive, ...), il reste pertinent de caractériser un algorithme par sa complexité.

Exemple : insertion successive de N éléments dans un tableau, initialement vide, en décalant tous les éléments suivants

- temps pour une affectation: T_1
- pire des cas : toutes les insertions au début du tableau
- nombre d'affectations du n -ième élément à insérer : n (càd $n-1$ décalages, 1 nouvelle affectation)
- temps total: $T_N = \sum_{n=0}^{N-1} n = \frac{(N-1)(N-2)}{2} = \frac{1}{2}N^2 + \dots \simeq N^2$
- donc complexité $O(N^2)$

Rem: lorsque l'on insère systématiquement en dernière position, la complexité est $O(N)$.

Question/Exercice : et si l'insertion se fait à une position aléatoire de la liste ?

Travaux pratiques : généralités et fonctionnement

Ces consignes resteront *a priori* identiques pour toutes les séances de TP.

Les travaux pratiques se déroulent dans les salles informatiques, sous Linux.

Attention, le choix de Linux est délibéré, l'accès que l'utilisation du terminal de commande donne à l'exécution du code permet de faciliter l'accès aux informations nécessaires.

Attention aussi : vous pouvez bien entendu effectuer les mêmes manipulations sur d'autres systèmes Linux, mais les résultats ne seront pas les mêmes. D'une part, une performance différente affectera les résultats quantitatifs mesurant la performance. D'autre part, et plus subtilement, les résultats peuvent aussi différer *qualitativement* ! Cela vient du fait que le compilateur peut différer, son comportement par défaut peut différer et, à la base de tout, l'architecture diffère nécessairement. Vous aurez été avertis !

Évaluation :

Le critère d'évaluation reposera davantage sur la démarche que sur vos résultats : peut-on comprendre votre démarche ? Pourquoi avez-vous conçu vos essais de telle manière ? Les paramètres, ont-ils été choisis de la bonne manière pour éviter des soucis ? Vos résultats, que signifient-ils ? Peut-on y croire ou pas ? Qu'est-ce qu'en apprend ? Etc.

A déposer :

- un document de synthèse **complet** : il doit documenter votre démarche et vos conclusions, permettre de comprendre et de reproduire vos résultats
 - en pdf (par exemple produit à partir de LibreOffice)
 - code et scripts éventuels (intégrés dans le texte, mais voir ci-dessous)
 - puis surtout : idée de la démarche, graphes produits, démarche d'exploitation des données, conclusion, appréciations critiques, etc.
- votre code, et d'éventuels scripts, en code 'source'

Délais :

Il vous est demandé de compléter votre compte-rendu au fur et à mesure, en ajoutant la suite avant chaque nouvelle séance, mais il restera amendable. La totalité du document sera évalué.

Travaux pratiques : Mesurer la performance

But : se familiariser avec les grandes lignes, et très probablement quelques subtilités, de la manière de quantifier la performance d’une opération simple dans un code.

Exercice : Coût des opérations élémentaires

On se propose de quantifier la performance pour des opérations simples, et en particulier de comparer la multiplication à la division, et si vous avez le temps à l’addition.

Implémentez un code effectuant N multiplications de deux nombres réels. Puis, en vous servant de la commande **time**, déterminez le temps de calcul nécessaire pour effectuer une multiplication.

Faites de même pour une division et concluez. Complétez par l’addition si vous en avez le temps.

Vous pouvez aussi contraster avec le résultat pour la multiplication de deux entiers : est-ce pareil ?

Quelques questions qui pourraient vous être utiles :

- combien de multiplications choisissez-vous d’effectuer? Dans l’idéal, et en pratique ? Pourquoi ?
- en répétant la mesure du temps d’exécution N fois, avec le même nombre N, que constatez-vous ?
- comment gérez-vous le problème qu’il y a un ‘overhead’ (surcout) dans le temps de calcul qui est associé au lancement de votre programme ?
- pensez à exclure des malentendus, comme ce qui peut se passer si vous effectuez la multiplication de deux constantes
- travaillez-vous avec des entiers ou avec des flots ? Que cela change-t-il ?

Une suggestion pratique :

Il est possible de recevoir un argument dans un program C/C++, ceci est illustré par le code fourni **commandlinearguments.C**. Après compilation, vous pouvez l’appeler comme ceci

```
> g++ -o cmdlargs.out commandlinearguments.C  
> ./cmdlargs.out testtesttest
```

et l’argument fourni (‘testtesttest’) est réceptionné par le code C comme chaîne de caractère.

Rem: En Fortran, le même type de mécanisme existe

(<https://gcc.gnu.org/onlinedocs/gfortran/GETARG.html> et

https://gcc.gnu.org/onlinedocs/gfortran/GET_005fCOMMAND_005fARGUMENT.html)

COMPILER AVANTAGEUSEMENT (PREMIERS ÉLÉMENTS)

On peut souvent obtenir des gains très importants
seulement en pilotant le compilateur correctement !

Illustration de flags de compilation

Ces ‘flags’ modifient la manière dont le code exécutable est produit à partir d’un seul et même fichier source.

Exemple :

```
/* programme compiltest.c */
#include <math.h>
#include <stdio.h>

main() {
    double x=0;
    for (long int i; i<10000000000; i++) {
        x += sinh(x);
    }
    printf("%lf" ,x);
}
```

En compilant cela par

```
g++ sometest.C -lm && time a.out
```

on obtient (à titre d’exemple) un temps ‘user’ de 5,366s, alors qu’en compilant ainsi

```
g++ -O2 sometest.C -lm && time a.out
```

cela devient 0.001s.

- gain d’un facteur de l’ordre de 5000 en performance (pour ce code, sur un système en particulier)
- typiquement les gains sont plus modestes mais pertinents
- c’est gratuit (!!!)
- sauf : temps de compilation risque d’augmenter

Illustration du principe

En fait, ce sont souvent autant des flags de **précompilation** :

- pre-compilateur modifie le code source.

Ex : le code **double y = x/2.0** peut être remplacé par **double y=x*0.5**, opération donc une opération moins coûteuse (a priori) !

Ex : le code **const x=42; y=x**2;** peut être remplacé par **x=42; y=1764;** et on peut ainsi économiser une multiplication

- en réalité il s'agit d'un ensemble d'interventions, de compilation et de pré-compilation : le comportement du compilateur est piloté par l'option de compilation **-O**
- le niveau de l'optimisation est choisi par le nombre spécifié :
 - **-O0** : aucune optimisation
 - **-O1** : un premier niveau d'optimisation, typiquement faciles à réaliser
 - **-O2** : optimisation avec interventions plus lourdes
 - **-O3** : optimisation très lourde
 - d'autres flags permettent d'aller au-delà ... (à suivre)
- la différence des niveaux se joue sur le type d'opérations que l'on tente d'améliorer, le nombre de lignes que l'on regarde 'en avance' pour identifier des optimisations à faire, etc

Attention !

Augmenter le niveau d'optimisation

- va typiquement (mais pas toujours !) optimiser la performance
- mais augmentera le temps de compilation
- et aussi la taille du binaire exécutable
- et potentiellement la mémoire vive nécessaire à l'exécution du programme

Attention, on parle aussi de '**l'agressivité**' de l'optimisation

- le code source sera modifié de manière de plus en plus importante
- en produisant, *en principe*, un exécutable équivalent
- **mais** : les optimisations très agressives peuvent entraîner des problèmes et subtilités, et occasionnellement cela conduit à des résultats numériques modifiés !!!

Règle de base :

- jusqu'à **-O2** l'optimisation est considérée 'safe'
- pour **-O3** les problèmes sont rares mais peuvent exister pour des cas particuliers
- le niveau **-Ofast** passe par-dessus certains standards stricts, en s'exposant à l'introduction d'erreurs par la compilation. **ATTENTION !!!** Ce niveau, et l'optimisation par d'autres flags 'sur mesure' (voir plus loin), sont susceptibles de poser problème ...

Rem : voir la documentation <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>

OPTIMISER SON CODE SOURCE

D'abord quelques rappels sur les variables, le typage et la gestion de la mémoire

Rôle du typage des variables (C) :

Compilation pour efficacité

- compilation exploite le typage:
 - le type de la variable est *figé* et donc connu au moment de compiler
 - comparer à Python (tout peut arriver ...)
- types de variables alphanumériques :
 - entiers : `int` et variantes : `short int`, `long int`, `unsigned int` ...
 - à virgule flottante : `float` (4 octets), `double` (8 octets)
 - et caractères: `char` (en réalité un `unsigned int`)
- Attention: valeurs minimales/maximales ne sont pas un standard, mais dépendent de l'architecture
 - voir `limits.h`

Coût des opérations simples :

Typiquement (mais pas toujours ...) :

addition, soustraction « multiplication « division « appel de fonctions (sqrt etc)

Attention: cela dépend de l'architecture, et les entiers peuvent se comporter différemment des réels

Ex : arithmétique d'entiers sur architecture Intel 64

- add/sub: 1 cycle CPU
- imul (integer multiply): 10 cycles
- idiv (integer division): 66–80 cycles(!)

Pour creuser la question:

→ <https://www.quora.com/Is-there-any-performance-difference-between-the-basic-operations-sum-subtraction-multiplication-division-when-the-computer-performs-them>

Surcoût d'un appel de fonctions

- surcoût inévitable (assembler : transmettre argument, chercher code de la fonction, etc)
- typiquement petit
- donc sans importance pour la plupart des fonctions : fonctions appelées rarement, fonctions lentes (i/o), etc

- potentiellement significatif sur les très petites fonctions

Mais, attention :

- souvent : performance/optimisation vs. lisibilité/structuration/modularité du code
- optimisations doivent cibler le code critique pour la performance (cf. Knuth !)

Ex : MD avec calcul des forces, images périodiques, etc

Donc, en pratique :

Pour les parties du code (uniquement !) qui sont critiques pour la performance

- remplacer une division par une multiplication, là où cela est possibles
Ex: $s = x * Li$, plutôt que $s = x / L$, (avec $Li = 1./L$ calculé une fois pour toutes)
- éviter les appels de fonctions inutiles
Ex: $x2 = x * x$, à la place de $x2 = \text{pow}(x, 2)$
- utiliser des *variables temporaires* si besoin
Ex: $r3 = r * r * r$; $r6 = r3 * r3$; à la place de $r6 = r * r * r * r * r * r$;

Mais, ***les calculs les plus rapides sont ceux qu'on évite de faire !***

→ d'autres économies peuvent être faites en amont

- conception de l'algorithme
→ complexité algorithmique
- choix du modèle (!)
→ le plus simple possible
→ numériquement simple

Ex: Lennard-Jones : pour calculer les forces et l'énergie, on n'a **jamais** besoin de calculer la distance r entre particules !

→ tout baser sur r^2

→ éviter ainsi d'appeler la fonction **sqrt** (souvent ... !!!)

Ex : potentiel de Lennard-Jones

code 'naif':

```
double u(double sig, double eps, double r) {
    return 4*eps*(pow(sig,12)/pow(r,12) - pow(sig,6)/pow(r,6) );
}
```

améliorations:

- pour commencer, appliquer la puissance au ratio, plutôt que de prendre le ratio des puissances
- mieux : éviter les facteurs **sig**, **eps** (*cf. variables adimensionnées !*)
- éviter l'appel de **pow()**
- éviter de calculer la racine
- utiliser des variables temporaires pour calculer les puissances en plusieurs étapes
- etc ...

Fonctions 'inline' : encore une astuce de compilation

But : déclarer des fonctions en évitant le surcoût lié à l'appel

Pour cela, l'idée est simple : il suffirait de remplacer, dans le code source, chaque appel de fonction par le code complet

Ex : puissance de six

```
double psix(double x) {
    double x2 = x*x; /*temporary variable*/
    return x2*x2*x2; }
sigma6 = psix(sigma);
alpha6 = psix(alpha);
```

pourrait être remplacé avantageusement (en ce qui concerne la performance) par

```
double tmp;
tmp=sigma*sigma;    sigma6=tmp*tmp*tmp;
tmp=alpha*alpha;    alpha6=tmp*tmp*tmp;
```

Inconvenient : multiplicité du code, mauvaise lisibilité, ...

On peut obtenir le même comportement par le mot clé **inline** !

```
inline double psix(double x) {
    double tmp x2 = x*x;
    return x2*x2*x2; }
```

```
sigma6 = psix(sigma);
alpha6 = psix(alpha);
```

À savoir :

- cela est utile *seulement* pour de très petites fonctions
- en réalité ils s'agit d'une **suggestion** pour le compilateur : c'est son algorithme qui décide si oui ou non la fonction est repliquée ou préservée en tant que telle

Rem: dans l'exemple, la présence d'une déclaration de variable pèse déjà lourdement en contre ...

Compromis:

```
inline double p3(double x2) {
    return x2*x2*x2; }
sigma6 = psix(sigma*sigma);
alpha6 = psix(alpha*alpha);
```

- par conséquent, le comportement précis dépend du compilateur
- et les flags d'optimisation renforcent (ou déjouent) la recommandation faite par le mot clé **inline**

Rem : il existe, en C/C++ un mécanisme plus rudimentaire appelé un **macro**, qui atteint le même but, mais de manière **certaine** :

```
#define sq(x) ( x*x )
x=sq(1989)
y=sq(1984)
```

Ici, aucune fonction n'est définie, mais le pré-processeur va réellement substituer **x*x** à chaque fois qu'il rencontre le code **sq(x)**. C'est très basique, mais ça marche bien (attention à qqs subtilités cependant, par exemple par rapport à la précedence de l'opérateur '*' une fois la substitution faite ... tester !

Ex: que se passe-t-il si l'on remplace, en tant que code, **sq(a+b)** → **a+b*a+b** ??? Comment arranger ce souci ?

Rem: il s'agit d'un exemple de directives de pré-processeur, qui propose d'autres mécanismes, e.g. pour une compilation conditionnelle selon si un certain flag est activé ou non (**#ifdef**).

Une autre astuce de pré-processeur: tests par compilation conditionnelle

Une autre situation qui arrive souvent : faire exécuter, ou non, une certaine partie du code

Mécanisme : **compilation conditionnelle**, gérée au niveau de la pré-compilation

Ex : afficher, mais seulement pour tester, une valeur

```
for (int i=0; i<10; i++) {
    int i2 = i*i;
    #ifdef TEST
    printf("i**2 : %d",i2);
    #endif
}
```

Le résultat à l'exécution dépend alors de la compilation, selon le paramètre précisé (ou non) :

```
> gcc moncode.c
> gcc -DTEST moncode.c
```

Ce mécanisme peut par exemple servir pour généraliser un code 2d à 3d, si cela est demandé (mais attention à tout maintenir de manière cohérente ...).

Quelques variantes :

- l'opposé (négation) :

```
#IFDEF ... #ENDIF
```

- traitement selon le cas (if .. else) :

```
#IFDEF ... #ELSE ... #ENDIF
```

- transmission d'une valeur, par ex 'LEVEL' (comme 'TEST' ci-dessus, par -DLEVEL)

```
#if LEVEL == 1 ... #endif
```

- etc

Se renseigner sur des directives de **préprocesseur**

Assertions

Un raccourci souvent bienvenu, justement pour sécuriser du code, reprend cette idée en forme d'une bibliothèque permettant de gérer des '**assertions**' ('affirmations'). Par exemple, le code

```
#include <stdio.h>
#include <assert.h>

void count(int N) {
    assert( N>0 );
    ... code of function ...;
}
```

permet de sécuriser la fonction : si jamais une valeur négative de N est passée, cela provoque l'arrêt du programme (avec un message d'erreur renvoyant à cette ligne de code).

En revanche, si cela est souvent justifié pour tester, ce ne peut pas être souhaitable en mode 'production', par exemple si cette fonction est appelée très fréquemment (ex: calcul des conditions périodiques aux bords). On peut alors d'en débarrasser simplement en précisant

```
gcc -DNDEBUG moncode.c
```

et toutes les assertions seront simplement éliminées par le pré-processeur !

C'est assez simple, n'affecte donc pas la performance lors de l'utilisation du code, et permet de détecter un certain nombre de soucis avant d'avoir très mal à la tête ...

Travaux pratiques : Compilation et optimisation du code source

But : se familiariser avec l'idée de pouvoir influencer sur la performance par des flags de compilation; comprendre l'impact de qqs flags simples sur la performance

ATTENTION : dans ce qui suit, il est parfois important de comparer à ce que fait le compilateur 'bêtement'. Or, il s'avère que le comportement de base peut déjà être configuré pour réaliser certaines optimisations (et ce comportement peut différer d'un système à l'autre, y compris pour un même compilateur ...). Il est donc important, lorsque l'on cherche une référence pour juger l'impact d'une optimisation précise, de compiler le code de référence en précisant l'option `-O0`, afin d'empêcher explicitement toute optimisation.

Exercice : coût d'opérations simples (sur les réels)

Reprendre votre analyse des opérations simples (addition, multiplication, division, éventuellement `sqrt(...)`), en testant les divers niveaux de compilation `-On`, en vous focalisant sur les nombres réels (`float` ou `double`). Conclusion ? Et si vous activez l'option `-ffast-math` à la compilation ?

Question extra : en vous appuyant sur la documentation, pouvez-vous indiquer, voire mettre en évidence, un exemple où le résultat numérique change suite à cette optimisation ?

Exercice : Interaction Lennard-Jones

On se donne les positions (x_1, y_1) et (x_2, y_2) de deux particules. L'interaction se fait selon le potentiel Lennard-Jones, en deux dimensions, tronqué à un rayon R_c et décalé pour assurer la continuité du potentiel. On a besoin d'une fonction.

```
const double Rc=2.5 /* global constant */

void interactions( double x1, double y1, double x2, double y2, \
                  double &u12, double &f12x, double &f12y ) {
    ...
}
```

qui calcule l'énergie d'interaction u_{12} ainsi que la force f_{12} de la particule no. 1 sur la particule no. 2 et modifie les valeurs définies par référence en fonction. Optimisez-la !

Cette question est donc un concours : quelle est la meilleure performance que vous pouvez obtenir sur les machines de la salle TP ?

Exercice (si vous avez le temps) : surcoût lié à l'appel de fonction / optimisation

Mettez en place une stratégie pour déterminer le surcoût lié à l'appel d'une fonction. A quel temps estimez-vous ce surcoût pour le système en salle de TP ? Est-elle significative ?

Rem : pour être à l'abris de chercher des erreurs inexistants, *il vaudra mieux empêcher le compilateur d'optimiser par défaut, en considérant que vous avez probablement voulu déclarer votre fonction comme **inline** : le flag de compilation **-fnoinline** (tout collé) vous permet d'empêcher cela.*

Rem : attention à ne pas généraliser – si vous appelez une fonction en C++ qui passe comme argument un objet, et vous passez cela par valeur, le surcoût peut être extrême, dû à l'opération de copie etc ... En Python, le surcoût pour appeler une fonction aura aussi tendance à être plus important.